

GLIDE

Arbeitsvorbereitung für Codex, Build, Release und Dokumentation

Windows · macOS · Branding · Packaging · QA · Signing · Stores · Automatisierung

Zweck dieses Dokuments

Dieses Dokument bündelt die Arbeitsvorbereitung für Glide auf Basis der vorhandenen Build- & Release-Anleitung, des Funktionskatalogs, der aktuellen Projektstruktur und einer Prüfung aktueller offizieller Anforderungen. Schwerpunkt ist nicht die fortgeschrittene Feature-Entwicklung, sondern die technische und organisatorische Vorbereitung für reproduzierbare Windows- und macOS-Releases sowie einen effizienten späteren Einsatz von ChatGPT Codex.

Dokumentversion 1.0 · Stand 31.08.2026

Arbeitsstatus: Planung / Pre-Release

Inhaltsverzeichnis

1. Management Summary
2. Ausgangslage und vorhandene Unterlagen
3. Was bereits richtig vorbereitet ist
4. Ergänzungen und offene Punkte der bisherigen Vorbereitung
5. Branding- und Icon-Vorbereitung
6. Empfohlene Gesamt- und Ordnerstruktur
7. Aufgabenverteilung: Codex vs. Benutzer
8. Empfohlene Codex-Arbeitspakete
9. Empfohlene AGENTS.md für Glide
10. Empfohlenes Dokumentationssystem
11. Zentrales Produkt- und Identitätsregister
12. Apple- und Microsoft-Accountentscheidungen
13. QA: reproduzierbares Testsystem
14. Referenz-Testdaten jetzt vorbereiten
15. Release-Artefakte und Versionierung
16. Security und Software-Supply-Chain
17. Empfohlener Grafik- und Asset-Workflow
18. Arbeitsvorbereitung während fehlender Codex-Credits
19. Verbindlicher Phasenplan
20. Strategische Leitprinzipien
21. Zielzustand für den späteren Release-Workflow
22. Fazit und nächste konkrete Schritte
23. Quellen und Prüfstand

Die Kapitel sind in Prozessreihenfolge angeordnet – von der Bestandsaufnahme und Arbeitsvorbereitung über Codex-Struktur, QA und Packaging bis zu Signing, Veröffentlichung und Automatisierung.

1. Management Summary

Gesamtbewertung. Die vorhandene Vorbereitung ist bereits weit fortgeschritten. Die bestehende Build- & Release-Anleitung deckt die komplette klassische Auslieferungskette ab: Projektstruktur, Branding, Windows- und macOS-Build, Installer/DMG, Signierung, Notarisierung, Benutzerdaten, Versionierung, QA, Stores und einen späteren KI-Arbeitsauftrag.

Der nächste Qualitätssprung besteht nicht darin, noch mehr einzelne Build-Befehle zu sammeln. Benötigt wird eine professionelle Projekt-Infrastruktur: ein Codex-fähiges Repository, verbindliche Produktmetadaten, automatisierbare Tests, klare Security-/Release-Regeln sowie eine saubere Trennung zwischen Sourcecode, Branding-Quellen, generierten Assets, Dokumentation und finalen Release-Artefakten.

Kernaussage

Für Glide sind keine grundlegenden EXE-/DMG-Schritte vergessen. Die wichtigste nächste Aufgabe ist, aus der vorhandenen Arbeitsordner-Struktur eine reproduzierbare Softwareprojekt- und Release-Struktur zu machen. Der funktionierende Python-Code sollte dabei zunächst nicht manuell umsortiert werden.

Die drei sinnvollsten Arbeiten während einer Codex-Pause sind:

1. Produkt-, Publisher- und Plattformscheidungen schriftlich fixieren.
2. Branding-Master einschließlich Apple-Layer sauber vorbereiten.
3. Workspace, Dokumentationsgerüst und Referenz-Testdaten vorbereiten – ohne den funktionierenden Python-Code vorschnell umzubauen.

2. Ausgangslage und vorhandene Unterlagen

2.1 Vorhandene Build- & Release-Anleitung

Die vorhandene Anleitung beschreibt bereits einen vollständigen Zielprozess von der Projektvorbereitung bis zur Veröffentlichung:

- Projekt und Stammdaten vorbereiten
- Branding und Icon-Master vorbereiten
- Windows mit PyInstaller bauen
- Windows-Installer mit Inno Setup erstellen
- Windows Code Signing
- macOS-App-Bundle erzeugen
- DMG erstellen
- Apple Developer ID, Signierung und Notarisierung
- Benutzerdaten, Einstellungen, Backups und Logs trennen
- Versionierung und Updatefähigkeit
- Clean-Machine- und Release-Tests
- Microsoft Store und Mac App Store
- spätere Automatisierung und KI-Unterstützung

Bewertung: Diese Basis ist solide und bleibt die zentrale technische Referenz. Die neuen Ergänzungen in diesem Dokument ersetzen sie nicht, sondern strukturieren und operationalisieren sie.

2.2 Funktionskatalog und Produktphilosophie

Der Funktionskatalog macht mehrere wichtige Produktgrenzen deutlich: Glide ist aktuell lokal ausgerichtet, benötigt kein Benutzerkonto und soll möglichst ohne externe Programmbibliotheken auskommen. Cloud-Synchronisation, serverbasierte Dienste und tiefgreifende Betriebssystemintegration sind bewusst spätere, aufwendigere Themen.

Konsequenz für Codex

Release-, Packaging- und Strukturarbeiten dürfen diese Produktphilosophie nicht stillschweigend verändern. Neue Runtime-Abhängigkeiten, Cloud-Dienste, Telemetrie, Kontensysteme oder Lizenzserver dürfen nicht „nebenbei“ eingeführt werden.

2.3 Aktuelle Arbeitsstruktur

 __pycache__		31.08.2026 13:06	Dateiordner	
 05_Probelisten_Testdaten		21.07.2026 11:48	Dateiordner	
 07_Python-Versionen		31.08.2026 16:02	Dateiordner	
 10_Dokumentation		31.08.2026 16:02	Dateiordner	
 100_Archiv		31.08.2026 12:08	Dateiordner	
 _test_glide_v250.py		31.08.2026 13:09	Python File	8 KB

Abbildung 1: Aktuelle sichtbare Glide-Arbeitsstruktur (Stand 31.08.2026).

Sichtbar sind unter anderem: **__pycache__**, **05_Probelisten_Testdaten**, **07_Python-Versionen**, **10_Dokumentation**, **100_Archiv** sowie **_test_glide_v250.py**. Für persönliche Arbeitsorganisation ist die Nummerierung nachvollziehbar. Für Git, Python-Paketierung, Tests, CI und Codex sollte der produktive Repository-Bereich jedoch konventioneller und strenger abgegrenzt werden.

3. Was bereits richtig vorbereitet ist

Bereich	Bewertung
Windows EXE	gut vorbereitet
Windows Installer	gut vorbereitet
macOS .app	gut vorbereitet
DMG	gut vorbereitet
Windows ICO	gut vorbereitet
macOS ICNS	gut vorbereitet
Apple Icon Composer	berücksichtigt
PyInstaller OneDir	richtig gewählt
Windows Signing	berücksichtigt
Apple Developer ID	berücksichtigt
Notarisierung	berücksichtigt
Benutzerdaten außerhalb Programmordner	berücksichtigt
Versionierung	berücksichtigt
Updatefähigkeit	berücksichtigt
Clean-Machine-Tests	berücksichtigt
Microsoft Store	grundsätzlich berücksichtigt
Mac App Store	grundsätzlich berücksichtigt
KI-Arbeitsauftrag	bereits vorhanden
Release-Checkliste	bereits vorhanden

Besonders sinnvoll ist die Entscheidung für PyInstaller OneDir als Ausgangspunkt. OneDir ist leichter zu diagnostizieren, startet in der Regel schneller und bildet eine robuste Basis für Installer. Unter macOS ist ein normales App-Bundle zusätzlich die sinnvollere Grundlage für signierte bzw. sandboxed Szenarien.

4. Ergänzungen und offene Punkte der bisherigen Vorbereitung

4.1 Microsoft Store: Silent Installation und unveränderliche Installer

Für eine klassische EXE/MSI-Einreichung sollte die vorhandene Anleitung um folgende Store-Anforderungen ergänzt werden:

- Silent Installation ohne Benutzerinteraktion muss funktionieren.
- Der Installer darf kein Webstub/Downloader sein, der wesentliche Bestandteile erst während des Setups nachlädt.
- Download-URLs sollten versioniert und dauerhaft eindeutig sein.
- Eine bereits eingereichte Datei hinter einer Download-URL darf nicht stillschweigend ersetzt werden.
- Installer und relevante PE-Binaries müssen digital signiert sein.
- Returncodes und Silent-Uninstall sollten für einen professionellen Win32-Release definiert und getestet werden.

```
Glide-Setup-2.5.0.exe /VERYSILENT /SUPPRESSMSGBOXES /NORESTART
```

4.2 Inno Setup: dauerhaft stabile AppId

Die Inno-Setup-AppId sollte einmalig erzeugt und anschließend über alle Glide-Versionen stabil gehalten werden. Sie identifiziert die Installation für Upgrade- und Deinstallationslogik. Eine versehentliche Änderung kann zu parallelen Installationen oder doppelten Deinstallationseinträgen führen.

```
AppId={{EINMALIG-GENERIERTE-STABILE-GUID}}
```

Wichtig ist die Trennung der Identitäten:

- Inno Setup AppId – Installationsidentität
- Windows AppUserModelID – Taskleisten-/Shell-Zuordnung
- macOS Bundle Identifier – Apple-Bundle-Identität
- Produktname – sichtbarer Marken-/Anwendungsname
- Versionsnummer – Release-Version

4.3 Windows Code Signing: SHA-256 und RFC-3161 Timestamping

Für öffentliche Windows-Releases sollte die Signaturstrategie nicht nur „signieren“, sondern auch den Zeitstempel festlegen:

```
SHA-256 Signatur
+
RFC-3161 Timestamp
```

Das Timestamping stellt sicher, dass eine korrekt signierte Veröffentlichung auch nach Ablauf des verwendeten Zertifikats weiterhin als zum Signierzeitpunkt gültig bewertet werden kann. Automatisierung sollte erst erfolgen, wenn der unsigned Build reproduzierbar und stabil funktioniert.

4.4 Apple Notarisierung: finalen Workflow präzisieren

Für den produktiven macOS-Release sollte der Ablauf klar in technische Prüfschritte zerlegt werden:

```
Glide.app bauen
↓
alle eingebetteten Binaries prüfen
↓
Glide.app mit Developer ID signieren
↓
Signatur verifizieren
↓
DMG erzeugen
↓
DMG signieren
↓
finale DMG mit notarytool an Apple senden
↓
Notarisierung prüfen
↓
Ticket staplen
↓
Gatekeeper-Test
```

Für aktuelle Apple-Workflows ist notarytool der relevante CLI-Weg. Hardened Runtime und korrekte Signierung der eingebetteten Bestandteile sind zentrale Voraussetzungen.

4.5 Marken-/Namensprüfung für „Glide“

Vor größerem Aufwand für Store-Grafiken, öffentliches Branding und Veröffentlichung sollte eine Namens-/Markenprüfung erfolgen. Es existiert bereits ein international aktives Softwareangebot unter dem Namen Glide/GlideOS. Das bedeutet nicht automatisch, dass die Bezeichnung für die eigene ToDo-App unzulässig ist, erhöht aber die Relevanz einer professionellen Prüfung.

- DPMA-Recherche
- EUIPO-Recherche
- relevante Waren-/Dienstleistungsklassen für Software
- Store-Namensverfügbarkeit
- Domain-/Webauftritt
- mögliche Verwechslungsgefahr

Hinweis

Die Markenprüfung in diesem Dokument ist eine Projekt- und Risikomanagement-Empfehlung, keine Rechtsberatung.

5. Branding- und Icon-Vorbereitung

Die bisherige Spezifikation mit einem hochwertigen 1024 × 1024 px Master, Vektorquelle und reproduzierbaren Exporten ist richtig. Technische Formate sollten aus einem kontrollierten Master erzeugt werden statt als voneinander unabhängige Masterdateien gepflegt zu werden.

5.1 Empfohlenes Masterpaket

```
Glide_Icon_Master.ai
Glide_Icon_Master.svg
Glide_Icon_Master.psd
Glide_Icon_Master_1024.png
```

```
Apple_Layers/
├─ 01_Background.svg
├─ 02_BaseShape.svg
├─ 03_MainSymbol.svg
├─ 04_Foreground.svg
└─ 05_Highlight.svg
```

Für Apple Icon Composer sollten getrennte, sinnvoll benannte Ebenen auf einem 1024 × 1024-Canvas vorbereitet werden. Vektor ist bevorzugt; Text sollte in Konturen umgewandelt werden. Systemmasken und bestimmte systemseitige Material-/Schatteneffekte sollten nicht unnötig in das Masterartwork eingebrannt werden.

5.2 Small-Size-Prüfung

Prüfgröße	Typische Verwendung
1024 px	Master / Store / große Vorschau
256 px	große Windows-Shell-Darstellung
128 px	hochauflösende Zwischengröße
64 px	Systemdarstellung
48 px	Explorer / Taskleiste bei Skalierung
32 px	Taskleiste / Start / Titelleiste
24 px	Taskleiste / Suche
16 px	Kontextmenü / kleine Shell-Ansicht

Für sehr kleine Größen kann eine optisch vereinfachte Variante sinnvoll sein, sofern feine Linien oder Details sonst verschwinden. Unter Windows werden zusätzlich Zwischenwerte durch Display Scaling relevant; der Multi-Resolution-ICO bleibt für klassische Win32-Apps dennoch der zentrale technische Container.

5.3 Trennung von Win32-ICO und späteren Store-/MSIX-Assets

Für eine klassische Windows-EXE reicht ein korrekt aufgebautes Multi-Resolution-ICO. Ein späteres MSIX-Paket benötigt dagegen ein separates Package-/Store-Assetset. Diese Assets sollten nicht ungeordnet mit den klassischen ICO-Ressourcen vermischt werden.

6. Empfohlene Gesamt- und Ordnerstruktur

Empfohlen ist eine Zweiteilung: ein persönlicher Workspace für Arbeitsmaterialien und ein technisch kontrolliertes Repository für Codex, Git, Tests, Packaging und Release-Automatisierung.

```
Glide_Workspace/
├── 00_Arbeitsvorbereitung/
│   ├── Entscheidungen/
│   ├── Research/
│   ├── Checklisten/
│   └── Notizen/
├── 01_Repository/
│   └── Glide/
│       ├── AGENTS.md
│       ├── README.md
│       ├── CHANGELOG.md
│       ├── LICENSE.md
│       ├── SECURITY.md
│       ├── .gitignore
│       ├── .gitattributes
│       ├── .editorconfig
│       ├── VERSION
│       ├── src/
│       │   └── glide/
│       │       ├── __init__.py
│       │       ├── __main__.py
│       │       ├── app.py
│       │       ├── version.py
│       │       ├── paths.py
│       │       └── resources/
│       ├── tests/
│       │   ├── unit/
│       │   ├── integration/
│       │   ├── smoke/
│       │   ├── packaging/
│       │   └── fixtures/
│       ├── assets/
│       │   ├── branding/
│       │   │   ├── master/
│       │   │   ├── windows/
│       │   │   └── macos/
│       │   └── apple-icon-composer/
│       ├── runtime/
│       └── installer/
│       ├── packaging/
│       │   ├── pyinstaller/Glide.spec
│       │   ├── windows/
│       │   │   ├── Glide.iss
│       │   │   ├── version_info.txt
│       │   │   └── signing/
│       │   └── macos/
│       │       ├── entitlements.plist
│       │       ├── dmg/
│       │       └── signing/
│       ├── scripts/
│       │   ├── build/
│       │   ├── assets/
│       │   ├── test/
│       │   ├── release/
│       │   └── verify/
│       ├── requirements/
│       │   ├── runtime.txt
│       │   ├── dev.txt
│       │   ├── build-windows.txt
│       │   └── build-macos.txt
│       └── docs/
```

```

├── 00_INDEX.md
├── 01_PRODUCT_CONSTRAINTS.md
├── 02_ARCHITECTURE.md
├── 03_BUILD_RELEASE.md
├── 04_BRANDING_ASSETS.md
├── 05_QA_TESTPLAN.md
├── 06_DATA_BACKUP_MIGRATION.md
├── 07_SECURITY_SIGNING.md
├── 08_STORE_PREPARATION.md
├── 09_SUPPORT_PRIVACY_LICENSE.md
├── 10_RELEASE_CHECKLIST.md
├── decisions/
├── exec-plans/
├── release/
│   ├── staging/
│   ├── manifests/
│   └── notes/
├── .github/workflows/
├── 20_Grafik_Master/
│   ├── Illustrator/
│   ├── Photoshop/
│   ├── Branding/
│   └── Arbeitsdateien/
├── 30_Release_Exports/
│   ├── Windows/
│   ├── macOS/
│   └── Hashes/
├── 40_Store_Material/
│   ├── Microsoft/
│   └── Apple/
├── 90_Testdaten_Extern/
└── 100_Archiv/

```

6.1 Zweck der Zweiteilung

Glide Workspace ist der persönliche Designer-/Projektarbeitsplatz. Dort dürfen AI-/PSD-Arbeitsdateien, Research, Screenshots, Entwürfe und Archive liegen.

01_Repository/Glide ist die technisch kontrollierte Softwarebasis. Nur dort arbeitet später Codex produktiv. Das reduziert das Risiko, dass alte Python-Kopien, Archive oder Teststände als aktueller Source interpretiert werden.

6.2 Umgang mit der aktuellen Struktur

Aktuell	Empfehlung
__pycache__	löschen bzw. per .gitignore ausschließen
05_Probelisten_Testdaten	automatisierte Fixtures nach tests/fixtures/; manuelle Testdaten nach 90_Testdaten_Extern
07_Python-Versionen	keine parallelen Sourcekopien als Dauerlösung; Build-Matrix und Versionsverwaltung dokumentieren
10_Dokumentation	technische Markdown-Doku nach repo/docs/; Word/PDF-Exporte können außerhalb bleiben
100_Archiv	außerhalb des produktiven Git-Repositories
_test_glide_v250.py	später nach Audit in stabile Source-Struktur überführen; Versionsnummer nicht dauerhaft im Source-Dateinamen tragen

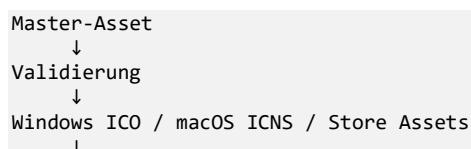
Wichtig

Die bestehende Python-Struktur jetzt nicht manuell umsortieren. Zuerst einen funktionsfähigen Ausgangsstand sichern, dann Codex ausschließlich lesend auditieren lassen und erst anschließend refactoren.

7. Aufgabenverteilung: Was Codex übernehmen kann und was beim Benutzer bleibt

Aufgabe	Codex	Benutzer
Repository analysieren	vollständig	Kontrolle
Import-/Dependency-Analyse	vollständig	–
Source-Struktur planen	vollständig	Freigabe
Dateien verschieben/refactoren	vollständig	Review
.gitignore / .gitattributes / .editorconfig	vollständig	–
AGENTS.md erstellen	vollständig	Regeln freigeben
Dokumentationsstruktur	vollständig	Inhalte freigeben
Tests erstellen	weitgehend	fachliche Prüfung
Smoke Tests	weitgehend	reale Endkontrolle
Datenpfade Windows/macOS	vollständig	–
Migrationen	weitgehend	Backup-/Datenfreigabe
PyInstaller Spec	vollständig	–
Windows Buildskripte	vollständig	ausführen/testen
Inno Setup Script	vollständig	Installer testen
Windows Version Resources	vollständig	Publisherdaten liefern
ICO erzeugen	technisch	Artwork liefern
ICNS erzeugen	technisch	Artwork liefern
Icon-Validator	vollständig	optische Kontrolle
DMG-Skript	vollständig	optische Kontrolle
DMG-Hintergrund technisch einbauen	vollständig	gestalten
Windows Signierskript	vollständig	Zertifikat/Account
Apple Signierskript	vollständig	Developer Account
Notarisierungsskript	vollständig	Credentials/Secrets
CI/CD	vollständig	Secrets hinterlegen
SHA-256 Manifest	vollständig	–
Release Notes	vollständig	Freigabe
Store-Texte	sehr gut	Freigabe
Store-Screenshots automatisieren	teilweise	finales Design
Datenschutzerklärung entwerfen	Entwurf	rechtliche Prüfung
Lizenz/EULA entwerfen	Entwurf	rechtliche Entscheidung
Markenprüfung recherchieren	Unterstützung	finale rechtliche Prüfung
Apple Developer Account	nein	Benutzer/Organisation
Microsoft Partner Account	nein	Benutzer/Organisation
Zertifikate anlegen/kaufen	Anleitung	Benutzer/Organisation
Finales Publishing	vorbereiten	Benutzer/Organisation

Rollenverständnis: Codex sollte für Glide primär als Software-/Release-Engineer eingesetzt werden. Das hochwertige Artwork und die Produktentscheidungen bleiben beim Designer bzw. Projektverantwortlichen; Codex übernimmt die technische Ableitung, Validierung, Builds, Tests und Automatisierung.



```

Build
↓
Installer / DMG
↓
Signing
↓
Tests
↓
Release Manifest

```

8. Empfohlene Codex-Arbeitspakete

Der bisherige umfassende KI-Arbeitsauftrag ist inhaltlich gut, sollte aber zur Kontrolle, Kredit-/Zeitersparnis und Risikoreduktion in kleine, klar abgrenzbare Tickets zerlegt werden.

C01 – Repository Audit

Analysiere das vollständige Glide-Projekt ausschließlich lesend.

Nimm keine Änderungen am Sourcecode oder an Dateien vor.

Erstelle:

1. vollständige Projektstruktur
2. Liste aller produktiv verwendeten Python-Dateien
3. Import- und Dependency-Analyse
4. Liste aller Runtime-Assets
5. Liste aller hartcodierten Dateipfade
6. Speicherorte der Benutzerdaten
7. mögliche Packaging-Probleme
8. mögliche macOS-/Windows-Inkompatibilitäten
9. Liste alter, redundanter oder möglicherweise archivierter Dateien
10. Risikoanalyse einer späteren Repository-Umstrukturierung

Unterscheide ausdrücklich zwischen produktivem Sourcecode, Testcode, Testdaten, Dokumentation, generierten Dateien und Archivmaterial.

Nimm keinerlei Refactoring vor.

C02 – Repository Foundation

Erstelle auf Basis des freigegebenen Audit-Berichts eine professionelle Repository-Struktur für Glide.

Ziele:

- konventionelle Python-Projektstruktur
- keine funktionalen Änderungen
- keine Änderung des Datenformats
- keine Änderung der Oberfläche
- keine neuen Runtime-Abhängigkeiten
- alle bisherigen Funktionen müssen weiter funktionieren

Erstelle zusätzlich:

```

.gitignore
.gitattributes
.editorconfig
AGENTS.md
README.md
VERSION
docs/00_INDEX.md

```

Dokumentiere jede Verschiebung und führe anschließend bestehende Tests bzw. Smoke Tests aus.

C03 – Release-Härtung

Prüfe Glide auf Release-Tauglichkeit.

Implementiere bzw. dokumentiere:

- plattformgerechte Datenpfade
- robuste Resource-Pfade für Source- und PyInstaller-Modus
- zentrale Versionsverwaltung
- Datenformat-/Schema-Version
- Backup vor Migrationen
- Logging
- Fehlerbehandlung

- Mehrfachstartverhalten
- Upgrade-Kompatibilität

Keine neuen Produktfunktionen hinzufügen.

C04 – Asset Pipeline

Erstelle für Glide eine reproduzierbare Asset-Pipeline.

Input:

assets/branding/master/

Output Windows:

- Glide.ico

Output macOS:

- Glide.icns

Store-Assets:

- getrennte Exportordner

Prüfe Abmessungen, Transparenz, Farbraum soweit automatisierbar, Dateinamen und benötigte Auflösungen.

Verändere das Artwork nicht gestalterisch.

C05 – Windows Build

Erstelle einen reproduzierbaren Windows-Releaseprozess für Glide.

Ziel:

Source → PyInstaller OneDir → Glide.exe → Inno Setup → Glide-Setup-x.y.z.exe

Berücksichtige:

- Glide.ico
- GUI-Fenstericon
- AppUserModelID
- Windows Version Resources
- stabile Inno-AppId
- Upgrade vorhandener Versionen
- Deinstallation
- Nutzerdatenerhalt
- Silent Install
- Silent Uninstall
- Logging
- Release-Dateinamen

C06 – macOS Build

Erstelle einen reproduzierbaren macOS-Releaseprozess.

Source → PyInstaller → Glide.app → Codesign-ready App Bundle → Glide.dmg

Berücksichtige:

- Glide.icns
- Bundle Identifier
- CFBundleVersion
- CFBundleShortVersionString
- Mindest-macOS-Version
- arm64/x86_64 Entscheidung
- Hardened Runtime
- Entitlements
- Developer-ID-Signing
- notarytool
- Stapling
- Gatekeeper-Verifikation

C07 – QA

Erstelle und automatisiere den Glide Release-Testplan.

Testbereiche:

- Unit
- Integration
- Smoke
- Packaging

- Installation
- Update
- Datenmigration
- Backup
- Restore
- Deinstallation
- Reinstallation
- Start nach Neustart
- fehlende Assets
- Windows DPI Scaling
- macOS Gatekeeper
- Signaturprüfung

Erzeuge einen maschinenlesbaren Testreport.

C08 – Release Automation

Automatisiere nach erfolgreicher Stabilisierung den Glide Release-Prozess.

Erzeuge:

- Buildskripte Windows
- Buildskripte macOS
- Assetgenerierung
- Testpipeline
- Versionsprüfung
- Release-Manifest
- SHA-256-Hashes
- Release Notes
- Changelog-Unterstützung

Signing Secrets dürfen niemals im Repository gespeichert werden.

9. Empfohlene AGENTS.md für Glide

AGENTS.md sollte als kurze Navigations- und Regeldatei dienen. Sie darf nicht zu einem zweiten, gigantischen Handbuch werden. Detailwissen gehört in docs/. Eine erste belastbare Version könnte so aussehen:

```
# Glide – Codex Instructions

## Product
Glide ist eine lokale Desktop-ToDo-Anwendung für Windows und macOS.

Aktuelle Grundprinzipien:
- lokale Datenspeicherung
- kein Benutzerkonto
- keine Cloudpflicht
- keine Netzwerkabhängigkeit für Kernfunktionen
- möglichst keine zusätzlichen Runtime-Abhängigkeiten
- Windows und macOS sind gleichwertige Zielplattformen

## Primary rule
Bestehende Funktionen und bestehende Nutzerdaten dürfen durch Build-, Packaging-, Dokumentations- oder
Strukturarbeiten nicht beschädigt werden.

## Source of truth
Produkt: docs/01_PRODUCT_CONSTRAINTS.md
Architektur: docs/02_ARCHITECTURE.md
Build und Release: docs/03_BUILD_RELEASE.md
Branding: docs/04_BRANDING_ASSETS.md
QA: docs/05_QA_TESTPLAN.md
Daten und Migration: docs/06_DATA_BACKUP_MIGRATION.md
Signing: docs/07_SECURITY_SIGNING.md
Stores: docs/08_STORE_PREPARATION.md
Release: docs/10_RELEASE_CHECKLIST.md

## Development rules
Vor strukturellen Änderungen:
1. aktuellen Zustand analysieren
2. Risiken dokumentieren
3. bestehende Tests ausführen
4. erst danach Dateien verändern

## Dependencies
Neue Runtime-Abhängigkeiten dürfen nicht ohne ausdrückliche Freigabe eingeführt werden.
Build- und Development-Abhängigkeiten dürfen separat verwaltet werden.
```

```

## User data
Benutzerdaten dürfen niemals im Installationsverzeichnis gespeichert werden.
Windows: %APPDATA%\Glide\
macOS: ~/Library/Application Support/Glide/

## Compatibility
Daten aus einer veröffentlichten älteren Glide-Version dürfen nicht ohne Migration unlesbar werden.
Vor Datenmigrationen muss ein Backup erstellt werden.

## Packaging
Windows-Builds werden auf Windows erstellt.
macOS-Builds werden auf macOS erstellt.
PyInstaller wird primär im OneDir-Modus verwendet.

## Security
Keine Passwörter, API-Keys, Zertifikate, PFX-Dateien, Provisioning-Profile oder Signing-Secrets
committen.

## Scope
Release-Arbeiten dürfen keine neuen Produktfunktionen implementieren, sofern diese nicht ausdrücklich
Bestandteil des Arbeitsauftrags sind.

## Completion
Nach jeder Änderung dokumentieren:
- geänderte Dateien
- Zweck der Änderung
- ausgeführte Tests
- Testergebnis
- offene Risiken

```

10. Empfohlenes Dokumentationssystem

Die vorhandene Word-Buildanleitung bleibt eine wichtige menschlich lesbare Arbeitsgrundlage. Für Codex und Git sollte parallel ein strukturiertes Markdown-System gepflegt werden.

Datei	Zweck
00_INDEX.md	Navigation und Dokumentationsindex
01_PRODUCT_CONSTRAINTS.md	Produktgrenzen und bewusste Nicht-Ziele
02_ARCHITECTURE.md	Source, Datenmodell, Module, Datenfluss
03_BUILD_RELEASE.md	technische Build-/Release-Anleitung
04_BRANDING_ASSETS.md	Logo-, Icon-, Store- und Exportregeln
05_QA_TESTPLAN.md	vollständiger Testplan
06_DATA_BACKUP_MIGRATION.md	Speicherorte, Schema, Backup, Restore
07_SECURITY_SIGNING.md	Zertifikate, Secrets, Hashes, Signing
08_STORE_PREPARATION.md	Apple-/Microsoft-Anforderungen
09_SUPPORT_PRIVACY_LICENSE.md	Datenschutz, Lizenz, Support
10_RELEASE_CHECKLIST.md	Go/No-Go-Checkliste
CHANGELOG.md	Änderungshistorie
SECURITY.md	Sicherheitsprinzipien
AGENTS.md	Codex-Navigation und dauerhaft geltende Regeln

10.1 PRODUCT_CONSTRAINTS.md als Schutz vor KI-Fehlentscheidungen

Diese Datei verhindert, dass Codex technisch plausible, aber strategisch unerwünschte Entscheidungen automatisch einführt – etwa SQLAlchemy, platformdirs, Cloudservices, Telemetrie oder ein Kontensystem.

```

# Product Constraints

## Current architecture
Glide ist Local First.
Kernfunktionen müssen ohne Internet funktionieren.
Keine Anmeldung ist erforderlich.
Benutzerdaten verlassen den Rechner nicht.

```

```
## Runtime dependencies
Neue externe Python-Runtime-Bibliotheken nur nach ausdrücklicher Entscheidung.
Buildtools wie PyInstaller zählen nicht als Runtime-Abhängigkeit.

## Future features
Folgende Funktionen dürfen bei Release-Härtung nicht vorweggenommen werden:
- Cloud Sync
- Lizenzserver
- Benutzerkonten
- Push-Infrastruktur
- Telemetrie
- externe Analytics

## Compatibility
Datenkompatibilität hat Priorität vor internem Refactoring.
```

11. Zentrales Produkt- und Identitätsregister

Vor dem ersten öffentlichen Release sollten die technischen Stammdaten nicht an mehreren Stellen verstreut gepflegt werden. Für die Arbeitsvorbereitung empfiehlt sich eine Datei wie:

```
00_Arbeitsvorbereitung/Entscheidungen/PRODUCT_IDENTITY.md
```

```
Produktname:
Glide
```

```
Langname:
Glide - Aufgaben und Listen
```

```
Aktuelle Version:
2.5.0
```

```
Publisher:
NOCH FESTLEGEN
```

```
Copyright:
NOCH FESTLEGEN
```

```
Support-E-Mail:
NOCH FESTLEGEN
```

```
Website:
NOCH FESTLEGEN
```

```
Datenschutz-URL:
NOCH FESTLEGEN
```

```
Lizenzmodell:
NOCH FESTLEGEN
```

```
Windows:
Ziel: Windows 10/11
Architektur: x64
```

```
Windows AppUserModelID:
NOCH FESTLEGEN
```

```
Inno Setup AppId:
NOCH FESTLEGEN
```

```
macOS Bundle Identifier:
NOCH FESTLEGEN
```

```
macOS Mindestversion:
NOCH FESTLEGEN
```

```
macOS Architektur:
NOCH FESTLEGEN
```

```
Release Channels:
internal
beta
stable
```

Datenformat-Version:
NOCH FESTLEGEN

12. Apple- und Microsoft-Accountentscheidungen früh treffen

12.1 Apple Developer Program

Vor öffentlichem Vertrieb sollte entschieden werden, ob als Einzelperson oder Organisation veröffentlicht wird. Bei einer Organisation sind Unternehmensdaten und je nach Apple-Prozess unter anderem eine D-U-N-S-Nummer sowie ein geeigneter geschäftlicher Web-/E-Mail-Auftritt relevant. Der veröffentlichte Verkäufer-/Entwicklername hängt von der gewählten Kontostruktur ab.

Diese Entscheidung sollte vor finaler Store-Gestaltung und vor dem Festlegen dauerhaft verwendeter Identitäten getroffen werden.

12.2 Microsoft Partner Center

Auch Microsoft unterscheidet Individual- und Company-Konten. Die Kontostruktur sollte frühzeitig festgelegt werden, da Publisher-Identität, Verifikation und spätere Store-Verwaltung davon abhängen. Account- und Identitätsentscheidungen sind nicht an Codex delegierbar.

13. QA: vom Checklistenpunkt zum reproduzierbaren Testsystem

Die vorhandene Release-Anleitung enthält bereits eine gute manuelle QA-Checkliste. Der nächste Schritt ist die Operationalisierung in vier technische Testebenen:

```
tests/
├─ unit/
├─ integration/
├─ smoke/
└─ packaging/
```

Ebene	Ziel
Unit	Einzelne Logikbausteine isoliert prüfen.
Integration	Zusammenspiel von Daten, Speicherung, Laden und Modulen prüfen.
Smoke	App starten und wichtigste Kernfunktionen in kurzer Sequenz prüfen.
Packaging	Tatsächlich gebaute EXE, .app, Installer bzw. DMG testen.

13.1 Release-Testmatrix

Test	Windows	macOS
Source-Start	Ja	Ja
PyInstaller Build	Ja	Ja
Clean Install	Ja	Ja
Erster Start	Ja	Ja
Daten anlegen	Ja	Ja
Neustart	Ja	Ja
Daten vorhanden	Ja	Ja
Update Alt → Neu	Ja	Ja
Backup	Ja	Ja
Restore	Ja	Ja
Uninstall	Ja	–
Nutzerdaten erhalten	Ja	Ja
Reinstall	Ja	Ja

Icon	Ja	Ja
DPI/Scaling	Ja	Retina/Skalierung
Signatur	Ja	Ja
Offline	Ja	Ja
Clean Machine ohne Python	Ja	Ja

14. Referenz-Testdaten jetzt vorbereiten

Ein kleines, bewusst konstruiertes Referenzset ist für spätere Regressionstests wertvoller als zufällig gewachsene Testlisten. Es sollte typische und problematische Zustände enthalten.

```

Liste 1
├─ Aufgabe normal
├─ Aufgabe erledigt
├─ Aufgabe mit Datum
├─ Aufgabe mit Priorität
└─ Unteraufgaben

Liste 2
├─ Sonderzeichen
├─ Umlaute
├─ sehr langer Text
└─ leere/ungewöhnliche Zustände

```

Die Referenzdaten sollten versioniert oder zumindest mit klarer Herkunft und Erwartungszustand dokumentiert werden. Für personenbezogene oder echte Unternehmensdaten sollten keine produktiven Inhalte in Testfixtures kopiert werden.

15. Release-Artefakte und Versionierung

Finale Veröffentlichungen sollten pro Version in einem unveränderlichen Release-Ordner zusammengefasst werden:

```

release/
├─ 2.5.0/
│   ├── windows/
│   │   └─ Glide-Setup-2.5.0-x64.exe
│   ├── macos/
│   │   └─ Glide-2.5.0-arm64.dmg
│   ├── RELEASE_NOTES.md
│   ├── release-manifest.json
│   └─ SHA256SUMS.txt
└─ {
    "product": "Glide",
    "version": "2.5.0",
    "channel": "stable",
    "build_date": "2026-08-31",
    "windows": {"architecture": "x64"},
    "macos": {"architecture": "arm64"}
  }

```

Ein final freigegebenes Release-Artefakt sollte danach nicht mehr unter identischem Dateinamen ausgetauscht werden. Korrekturen erhalten eine neue Version bzw. Buildnummer.

16. Security und Software-Supply-Chain

```

Pinned Build-Dependencies
  ↓
Clean Build Environment
  ↓
Build
  ↓
Malware Scan
  ↓
Signing
  ↓
Signature Verification

```



```

↓
SHA-256
↓
Release Manifest

```

Besonders wichtig ist die Trennung von Build-Konfiguration und geheimen Signierdaten. Folgende Dateien bzw. Geheimnisse dürfen niemals ins Repository:

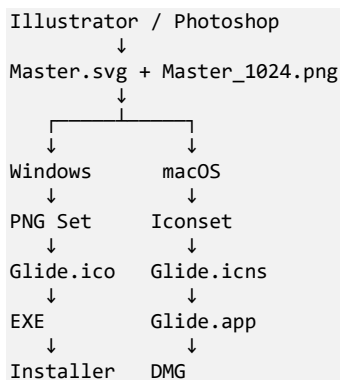
- *.pfx
- *.p12
- *.key
- *.pem
- Apple API Keys
- Notary Credentials
- Signing Tokens
- Passwörter und Recovery-Codes

Repository-Regel

Signing-Secrets gehören in Betriebssystem-Keychains, Secret Stores oder CI-Secrets. Das Repository enthält ausschließlich Skripte und Konfiguration, niemals private Schlüssel.

17. Empfohlener Grafik- und Asset-Workflow

Die grafische Arbeit bleibt in Illustrator/Photoshop. Die technische Größen- und Formatvielfalt wird anschließend reproduzierbar erzeugt.



Das Artwork wird nicht durch Codex „verbessert“ oder gestalterisch verändert. Codex validiert Größen, Dateiformate und Exportvollständigkeit und automatisiert die Ableitung technischer Assets.

18. Arbeitsvorbereitung während fehlender Codex-Credits

18.1 Priorität 1 – funktionierenden Stand einfrieren

Eine vollständige Sicherheitskopie des aktuell funktionierenden Stands erstellen. Beispiel:

```
Glide_PreRelease_2.5.0_2026-08-31
```

Noch keine strukturellen Refactorings durchführen.

18.2 Priorität 2 – Produktidentität festlegen

- Publisher
- Copyright
- Supportkontakt
- Website
- Bundle Identifier
- Windows AppUserModelID
- Inno Setup AppId

- macOS Mindestversion
- macOS Architektur
- Windows Architektur
- Lizenzmodell
- Release-Kanäle
- Datenformat-Version

18.3 Priorität 3 – Branding-Master vorbereiten

```
Glide_Icon_Master.ai
Glide_Icon_Master.svg
Glide_Icon_Master.psd
Glide_Icon_Master_1024.png
```

```
Apple-Layer:
01_Background.svg
02_Base.svg
03_Symbol.svg
04_Foreground.svg
```

Anschließend optische Tests in 1024, 256, 128, 64, 48, 32, 24 und 16 px durchführen.

18.4 Priorität 4 – Referenz-Testdaten definieren

Mindestens typische, erledigte, terminierte, priorisierte, verschachtelte und ungewöhnliche Einträge vorbereiten. Sonderzeichen, Umlaute, sehr lange Texte und leere Zustände bewusst berücksichtigen.

18.5 Priorität 5 – Dokumentationsgerüst anlegen

```
AGENTS.md
docs/
├── 00_INDEX.md
├── 01_PRODUCT_CONSTRAINTS.md
├── 02_ARCHITECTURE.md
├── 03_BUILD_RELEASE.md
├── 04_BRANDING_ASSETS.md
├── 05_QA_TESTPLAN.md
├── 06_DATA_BACKUP_MIGRATION.md
├── 07_SECURITY_SIGNING.md
├── 08_STORE_PREPARATION.md
└── 10_RELEASE_CHECKLIST.md
```

18.6 Priorität 6 – Store-Grafiken noch nicht final produzieren

Store-Screenshots sollten erst erstellt werden, wenn UI und Release-Build nahezu eingefroren sind. So werden unnötige Mehrfachproduktionen vermieden. Für einen späteren Mac-App-Store-Release sind unter anderem 16:10-Screenshotformate relevant; die jeweils aktuellen Spezifikationen sollten unmittelbar vor der Einreichung erneut geprüft werden.

19. Verbindlicher Phasenplan

Phase	Verantwortung	Ergebnis
Phase 0 – Arbeitsvorbereitung	Benutzer	Produktentscheidungen vollständig; Branding-Master vollständig; Testdaten vorbereitet; Projekt gesichert; Dokumentationsgerüst vorhanden
Phase 1 – Audit	Codex	AUDIT_REPORT.md; DEPENDENCY_REPORT.md; FILE_MAP.md; PACKAGING_RISKS.md – ohne Änderungen am Sourcecode
Phase 2 – Repository-Härtung	Codex	saubere Projektstruktur; Git-Basis; AGENTS.md; Tests; stabile Versionsquelle; keine Funktionsänderung
Phase 3 – Asset-Pipeline	Benutzer + Codex	Benutzer liefert Artwork; Codex erzeugt ICO/ICNS, validiert und exportiert
Phase 4 – Windows Release	Codex + Benutzer	Glide.exe und Glide-Setup-x.y.z.exe; zunächst

Candidate		unsigned testen
Phase 5 – macOS Release Candidate	Codex + Benutzer	Glide.app und Glide-x.y.z.dmg; zunächst lokal stabilisieren
Phase 6 – QA	Codex + Benutzer	Install, Update, Backup, Restore, Uninstall/Reinstall, Clean Machine, Scaling, Assets
Phase 7 – Signing	Benutzer + Codex	Windows Authenticode, Apple Developer ID, Notarisierung; Secrets außerhalb Repository
Phase 8 – Direct Release	Benutzer	signierte EXE/DMG direkt veröffentlichen; reale Nutzer-/Clean-Machine-Erfahrung sammeln
Phase 9 – Stores	Benutzer + Codex	Microsoft Store zuerst; Mac App Store anschließend mit Sandbox-/Entitlement-Prüfung
Phase 10 – Automatisierung	Codex	Build → Test → Sign → Package → Verify → Hash → Release weitgehend skript-/CI-gestützt

20. Strategische Leitprinzipien

Automatisierungsprinzip

Funktioniert manuell → ist dokumentiert → ist reproduzierbar → ist getestet → wird automatisiert.

Die umgekehrte Reihenfolge – zuerst komplexe Automatisierung zu bauen und anschließend zu hoffen, dass der technische Prozess stimmt – ist insbesondere bei PyInstaller, Installer, Signierung, Notarisierung und Datenmigration unnötig riskant.

Zweites Leitprinzip: Keine strukturelle Änderung ohne reproduzierbaren Ausgangstest. Vor dem Verschieben oder Refactoring produktiver Dateien soll Codex zunächst die tatsächlichen Imports, Pfade, Assets und Datenzugriffe des aktuellen Projekts analysieren.

Drittes Leitprinzip: Feature-Entwicklung und Release-Härtung getrennt behandeln. Neue Produktfunktionen dürfen nicht stillschweigend in Packaging-/Build-Tickets hineinrutschen.

21. Zielzustand für den späteren Release-Workflow

```

Benutzer:
Version und Design freigeben
↓
Codex:
Code analysieren
Tests ausführen
Version prüfen/setzen
Assets validieren
↓
Windows-Workflow:
EXE → Installer → Tests
↓
macOS-Workflow:
.app → DMG → Tests
↓
Release-Workflow:
Hashes → Manifest → Release Notes
↓
Benutzer:
Finale Freigabe
↓
Signing / Veröffentlichung

```

Dieser Zielzustand reduziert manuelle Wiederholungsarbeit, ohne die finale fachliche, gestalterische, rechtliche oder accountgebundene Freigabe aus der Hand zu geben.

22. Fazit und nächste konkrete Schritte

Die bestehende Vorbereitung ist technisch belastbar. Es fehlt kein grundlegender EXE-/DMG-Baustein. Die offenen Themen liegen überwiegend in Repository-Governance, formaler Produktidentität, Testautomatisierung, Security, Store-Details und der technischen Trennung zwischen Arbeitsmaterial und produktiver Codebasis.

Empfohlene Reihenfolge ab jetzt:

1. Arbeitsstand sichern und nicht manuell refactoren.
2. Produktidentität und Plattformscheidungen ausfüllen.
3. Icon-/Branding-Master und Apple-Layer vorbereiten.
4. Referenz-Testdaten definieren.
5. Workspace- und Dokumentationsstruktur anlegen.
6. Bei wieder verfügbaren Codex-Credits mit C01 – Repository Audit starten.
7. Erst nach freigegebenem Audit produktive Dateien umstrukturieren.
8. Windows und macOS zunächst unsigned reproduzierbar bauen und testen.
9. Signing und Store-Veröffentlichung erst auf einen stabilen Releaseprozess setzen.

Startpunkt für Codex

Der erste neue Codex-Auftrag sollte C01 – Repository Audit sein. Er ist ausdrücklich lesend und verändert keine Datei. Das schafft eine belastbare Grundlage für alle danach folgenden Refactorings und Release-Automatisierungen.

23. Quellen und Prüfstand

Interne Projektgrundlagen:

- Glide – Funktionskatalog, hochgeladene Projektunterlage, Stand 31.08.2026.
- Glide – Build-, Packaging-, Icon- und Release-Anleitung, Dokumentversion 1.0, Stand 31.08.2026.
- Aktuelle sichtbare Ordnerstruktur aus dem vom Benutzer bereitgestellten Screenshot, Stand 31.08.2026.

Externe offizielle bzw. primäre Referenzen, die für die Ergänzungen herangezogen wurden:

- [OpenAI – Introducing the Codex app](#)
- [OpenAI – Introducing Codex](#)
- [OpenAI – Harness engineering](#)
- [PyInstaller – Operating Mode](#)
- [Microsoft Learn – App package requirements for MSI/EXE apps](#)
- [Microsoft Learn – App icon construction](#)
- [Inno Setup Help – Appld](#)
- [Microsoft Learn – Artifact Signing certificate management](#)
- [Microsoft Learn – Artifact Signing integrations](#)
- [Apple Developer – Notarizing macOS software before distribution](#)
- [Apple Developer – Icon Composer](#)
- [Apple Developer – Program enrollment](#)
- [Apple Developer – Program membership/enrollment](#)
- [Apple Developer – App Store Connect screenshot specifications](#)
- [Apple Developer – macOS distribution](#)
- [Microsoft Learn – Open a developer account](#)
- [Glide – vorhandenes Softwareangebot zur Namens-/Markenrisikoprüfung](#)

Prüfstand: 31.08.2026. Store-, Signierungs- und Plattformanforderungen können sich ändern; vor einer tatsächlichen Einreichung oder Zertifikatsbeschaffung sollten die offiziellen Anforderungen erneut geprüft werden.